BASE / basemode: memory delivery, current setup and correction tests

Researched 2026-11-02.

A second brain delivered through hooks

BASE, now branded basemode, combines a local knowledge graph with code structure, rules, decisions and targeted context delivery. The reel's central idea is that an agent benefits when a relevant rule arrives before an action, rather than depending on the model remembering to search. The official site (https://www.basemode.ai/) describes that approach; its productivity comparisons remain vendor claims rather than a controlled result from Kiln. Independently checked November 2, 2026.

The current source README (https://github.com/ChristopherKahler/base) identifies a Rust binary and Claude Code hook integration. The installation documentation (https://docs.basemode.ai/install) explicitly says the installer wires Claude Code, while Codex and Antigravity require manual connection. Host-independent files do not imply automatic support for every host. The source lists FSL-1.1-ALv2, so do not describe it as having the same license as every open-source memory tool.

Current setup route

Install Claude Code first. Inspect the upstream installer before running its documented Windows PowerShell bootstrap:

```
irm https://raw.githubusercontent.com/ChristopherKahler/base/main/install.ps1 | iex
```

That downloads a release binary and invokes `base install`; `base install` by itself is not a download command for an absent executable. In a chosen test workspace:

```
base scaffold
base doctor
base commands list
```

The installer changes global Claude configuration, appends its section to the global instruction file and installs a bundled skill. Inspect the documented changes first. A manually installed binary can use `base install --skip-hooks` to start as an ordinary CLI. The documentation says `base uninstall` removes the integration but leaves the global memory directory, so uninstalling is not a complete data-erasure operation.

Test relevance and corrections

Use a disposable repository containing a billing module and an unrelated documentation module. Teach one fictional rule through the command supported in the current README:

```
base rule add --domain myapp --text "Migrations always run through the CLI, never raw SQL"
base context "How should I run this myapp migration?"
```

Then open a fresh session, ask a relevant question and inspect whether the rule arrives and changes the proposed action. Ask an unrelated question to test excessive injection. Correct the rule and check that the old decision no longer governs the answer. Repeat after a restart and after editing a mapped code file. Capture the injected evidence and final answer separately.

Compare tools fairly

BASE emphasizes delivery at action time; OptMem emphasizes bounded memory navigation, and an Obsidian wiki emphasizes editable source-backed notes. A code graph describes structure and does not prove semantic understanding. These tools are related but independently useful, so this source stays a separate tile. Compare them on the same task, model, prompt and evidence; include setup effort, incorrect retrieval and stale facts along with token use.

Reusable prompt and project

```
Evaluate a memory integration with ten fictional repository questions.
Include relevant and irrelevant prompts, a corrected rule and a renamed
file. Record exactly what context arrived, whether it cited current facts,
and whether the final answer followed the right rule. Keep the model and
task fixed between methods. Report missing measurements as unknown; do
not convert a vendor demonstration into a productivity benchmark.
```

Build a context-delivery scorecard with expected rule, injected context, stale-rule errors and task correctness. A blank template is included. Begin with one test workspace; these commands are documented, not executed by Kiln.

Original sources

- https://www.instagram.com/reel/De95N60JYG2/